

OpenVisionSearch

Open-source, self-hosted image similarity search API

Version 3 - Goal, Scope, Usage, Storage, Model Selection, and Agent Skill

Topic	Decision
Product type	Free open-source self-hosted visual search API
Default stack	FastAPI + OpenCLIP + Qdrant + Docker Compose
Default storage rule	Store vectors and metadata, not original images
Supported inputs	URL, upload, path, folder, base64; signed URLs for S3/R2
Future model design	Selectable embedding providers per collection

1. Product Mission

OpenVisionSearch is a free, open-source, self-hosted image similarity search service for developers. It lets teams install the service inside their own environment, index images from multiple sources, and call HTTP endpoints to find visually similar images.

The service is not primarily an image storage system. It is a visual search engine. By default it stores vectors and metadata, not original images.

Developer images -> OpenCLIP embeddings -> Qdrant vectors
Query image -> same model -> nearest-neighbor search -> similar image results

2. Target Users and Use Cases

User Type	Use Case
Backend developers	Add image search to existing apps through REST API
E-commerce teams	Search similar products by image
Marketplace teams	Find similar listings and near duplicates
CMS developers	Search media libraries visually
Internal tool teams	Add visual lookup without cloud AI lock-in

3. Core Image Storage Principle

Default behavior should be: read the image, create an embedding vector, store the vector and metadata, then discard the temporary image. This keeps the project lightweight, privacy-friendly, and easy to self-host.

Read image -> create vector -> store vector + metadata -> discard temporary image

Search responses should return the image ID, score, stable display image URL, and metadata, so the developer app can render the original image from its own storage/CDN.

4. Image Sources and Third-party Storage

Source	v0.1	Notes
url	Yes	Public URL, CDN URL, S3/R2 signed URL
upload	Yes	Multipart file upload
path	Yes	Path visible inside the service/container only
folder	Yes	Batch index a mounted folder
base64	Yes	Useful for mobile/frontend systems
s3 direct	Later	Bucket/key access with credentials
r2/minio/wasabi direct	Later	S3-compatible connectors

S3/R2/Private Storage - v0.1 Strategy

Use signed URLs first. The developer backend generates a temporary signed URL, sends it to OpenVisionSearch, and the service downloads it before expiry. OpenVisionSearch then stores only vector and stable metadata.

Team backend -> generate signed URL
OpenVisionSearch -> download temporarily -> vectorize -> store vector + metadata
Do not store expiring signed URL as permanent display URL

For private storage, store stable fields such as `display_image_url`, `storage_provider`, `bucket`, and `object_key` in metadata.

5. Search Algorithm

OpenVisionSearch uses image embeddings for similarity search. The model converts each image into a vector. Similar-looking or semantically related images produce vectors that are close together. Qdrant searches for the closest stored vectors.

Image -> OpenCLIP image encoder -> vector embedding -> Qdrant nearest-neighbor search -> ranked results

This is different from exact duplicate search. A red sneaker and white sneaker may be similar even if they are not the same image. Later, duplicate mode can be added with perceptual hashing.

6. Model Strategy and Future Selection

v0.1 should use OpenCLIP only, but the code should be designed with model providers from the beginning. Future users can select a model per collection.

Provider	Status	Purpose
openclip	v0.1	Default general image embeddings
custom	Future	Your own trained model
siglip	Future	Alternative embedding provider
dinov2	Future	Alternative visual embedding model

Important: one collection must use one embedding model. Do not mix vectors from different models in the same collection. Changing the model requires re-indexing.

7. Technical Stack

Layer	Choice	Reason
API	FastAPI	Fast, clean, automatic OpenAPI docs
Embedding	OpenCLIP	Free/open-source friendly image embeddings
Vector DB	Qdrant	Docker-friendly vector search with payloads
Image processing	Pillow	Common Python image handling
HTTP download	httpx	Async image downloads
Config	Pydantic Settings	Clean environment-based config
Tests	Pytest	Standard Python testing

8. Project Structure

```

open-vision-search/
├── app/
│   ├── main.py
│   ├── api/routes/{health,collections,index,search,images,models}.py
│   ├── core/{config,constants,errors,security}.py
│   ├── schemas/{collection,image_source,index,search,model,common}.py
│   ├── services/{image_loader,indexing_service,search_service,collection_service,vector_service,storage_service}.py
│   ├── embedding/{base,registry,manager}.py
│   ├── embedding/providers/{openclip_provider,custom_provider}.py
│   ├── providers/qdrant_provider.py
│   ├── providers/storage/{local_provider,s3_provider}.py
│   └── utils/{image_utils,path_utils,url_utils,id_utils}.py

```

```
docs/
examples/{curl,python,javascript,php,go}/
tests/
data/{images,uploads}/
Dockerfile
docker-compose.yml
requirements.txt
.env.example
README.md
```

9. API Scope

```
GET /health
POST /collections
GET /collections
GET /collections/{collection_name}/stats
DELETE /collections/{collection_name}
GET /models
POST /collections/{collection_name}/index
POST /collections/{collection_name}/index/upload
POST /collections/{collection_name}/index/folder
POST /collections/{collection_name}/search
POST /collections/{collection_name}/search/upload
POST /collections/{collection_name}/search/batch
GET /collections/{collection_name}/images/{image_id}
DELETE /collections/{collection_name}/images/{image_id}
```

10. Security Requirements

Area	Required Controls
Path	Allow only inside ALLOWED_IMAGE_ROOT; block ../ traversal; reject non-image files
URL	Timeouts; max redirects; max file size; content-type validation; block private IPs by default
Upload/base64	Max size; allow jpg/jpeg/png/webp; verify with Pillow; convert to RGB
Model	Store model info per collection; do not mix model vector spaces

11. Roadmap

Version	Scope
v0.1 Core MVP	FastAPI, Qdrant, OpenCLIP, collections, index URL/path/folder/base64/upload, search, Docker, README
v0.2 DX	Examples for curl/Python/JS/PHP/Go, metadata filtering, batch improvements, multiple-image search
v0.3 Production	API keys, background jobs, indexing progress, optional storage drivers, direct S3/R2/MinIO, GPU mode
v1.0 Stable	Stable API, full docs, tests, provider/plugin system, model selection, production deployment guide

12. Implementation Order

1. FastAPI app + /health
2. Dockerfile + docker-compose with Qdrant
3. Pydantic Settings config
4. Schemas for collection/model/image source
5. Qdrant provider
6. Collection service
7. Model registry with OpenCLIP only
8. Image loader for URL/path/base64/upload
9. Security checks for URL/path/upload
10. OpenCLIP provider
11. Indexing service and endpoint
12. Search service and endpoint
13. Folder indexing
14. Stats/delete endpoints

- 15. Tests
- 16. README and examples

13. Definition of Done for v0.1

docker compose up works.
/docs opens successfully.
A developer can create a collection.
A developer can index URL, path, folder, base64, and upload images.
A developer can search URL, path, base64, and upload images.
Results return id, score, display_image_url, and metadata.
Qdrant stores vectors and payloads.
Original images are not stored by default.
README explains signed URL strategy for S3/R2/private storage.
Tests cover path safety and invalid images.